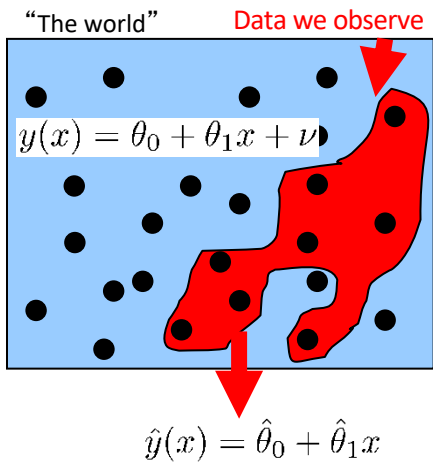
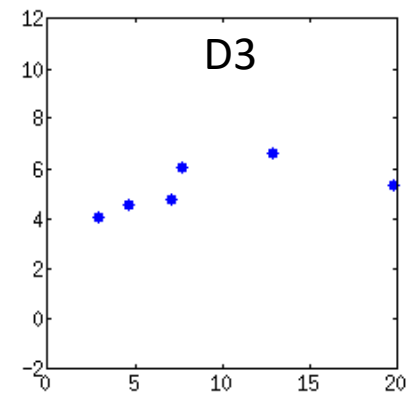
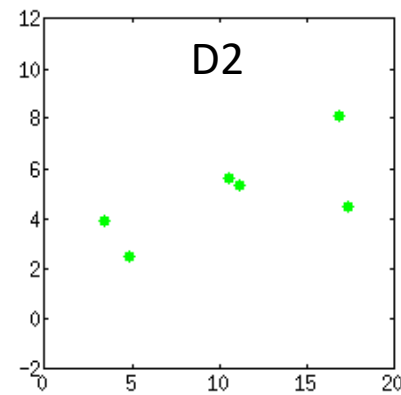
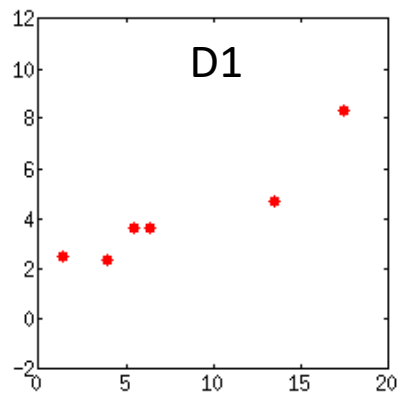


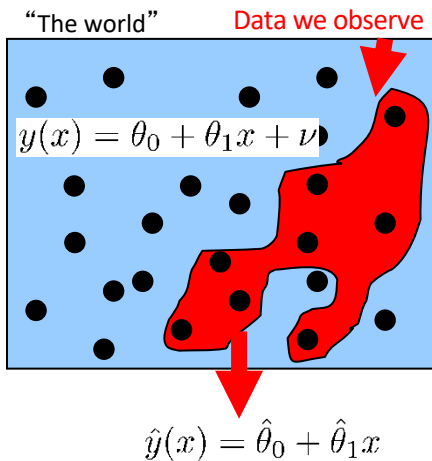
Bias & variance



Three different possible data sets:

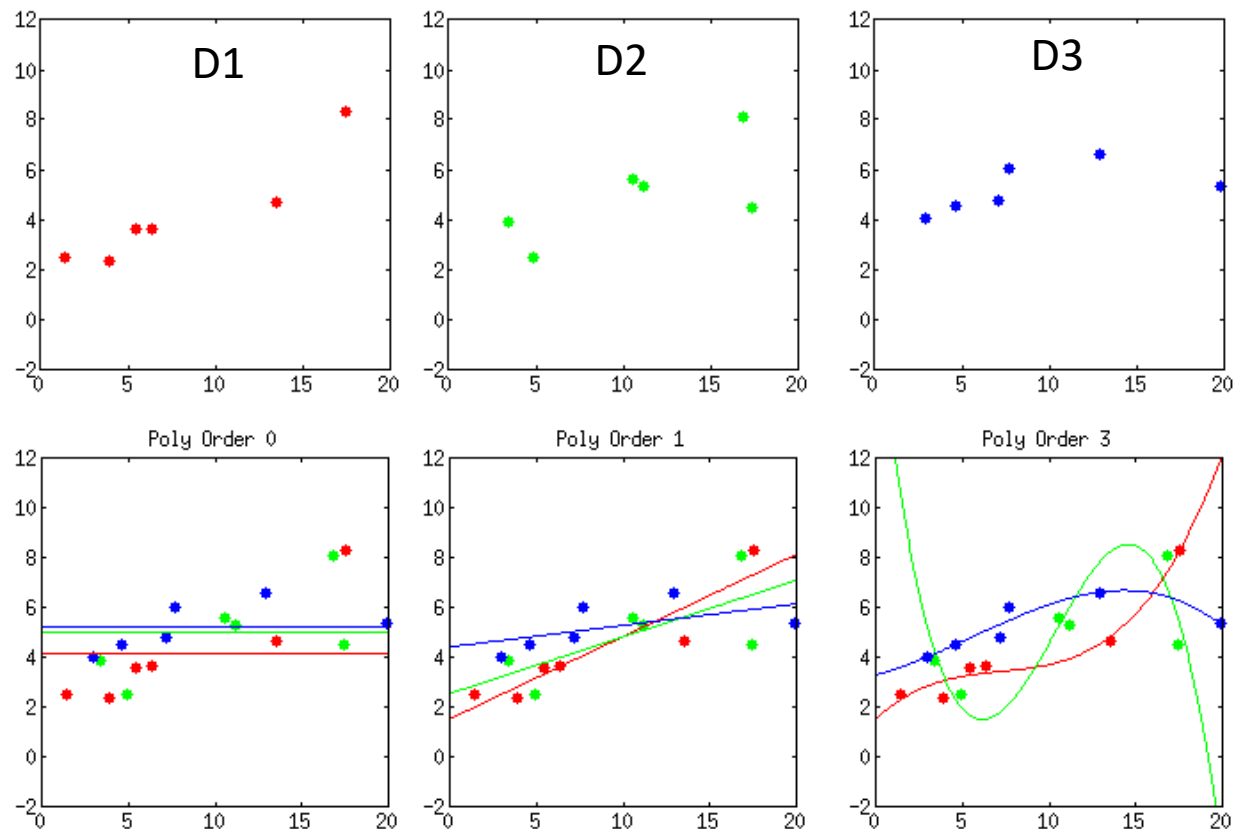


Bias & variance



Each would give different predictors for any polynomial degree:
The more complex, the more diverse!

Three different possible data sets:



Detecting overfitting

High variance = overfitting effect

- Do better on training data than on future data
- Need to choose the “right” complexity

One solution: “Hold-out” data

Separate our data into two sets

- Training
- Test/validation

Learn only on training data

Use test/validation data to estimate generalization quality

- Model selection

All good competitions use this formulation

- Often multiple splits: one by judges, then another by you

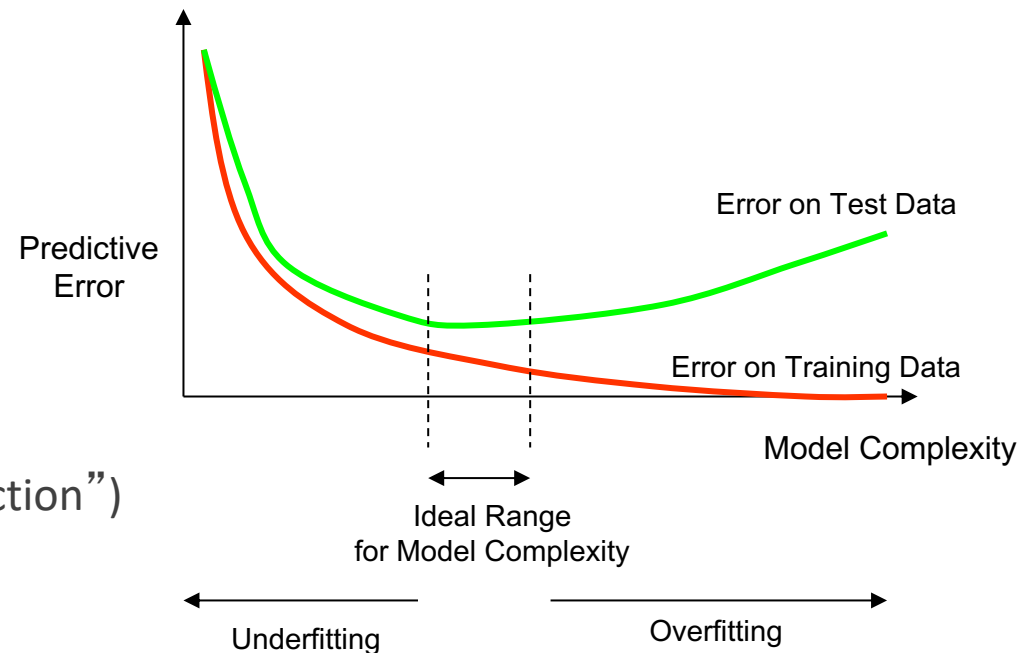
What to do about under/overfitting?

Ways to increase complexity?

- Add features, parameters
- We'll see more...

Ways to decrease complexity?

- Remove features (“feature selection”)
- Force model to underperform
 - Goal: “Fail to fully memorize data”
 - Partial training (“early stopping”)
 - Regularization (next)



Example: model selection in linear regression

Linear model, two data points

Quadratic model, two data points?

- Infinitely many settings with zero error
- How to choose among them?

Higher order coefficients = 0?

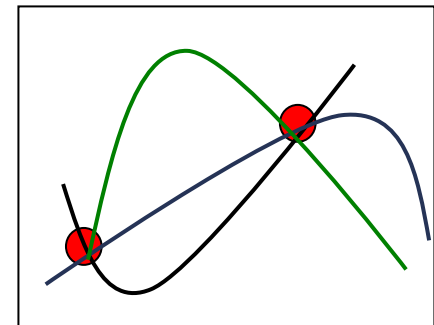
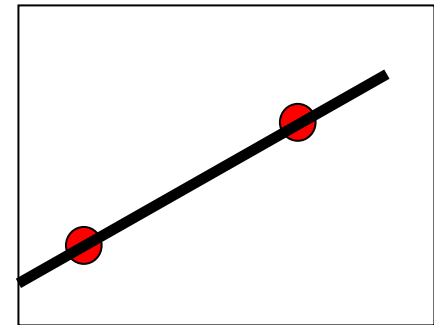
- Too specific to polynomials...

Could choose e.g. minimum magnitude:

$$\min \underline{\theta} \underline{\theta}^T \quad s.t. \quad J(\underline{\theta}) = 0$$

A type of bias: tells us which models to prefer

- Understand mechanism later



Regularization

Can modify cost function to add “preference” for certain parameter values

$$J(\underline{\theta}) = \frac{1}{2} (\underline{y} - \underline{\theta} \underline{X}^T) \cdot (\underline{y} - \underline{\theta} \underline{X}^T)^T + \alpha \underline{\theta} \underline{\theta}^T$$

L₂ penalty:
“Ridge regression”

New solution (derive the same way)

$$\underline{\theta} = \underline{y} \underline{X} (\underline{X}^T \underline{X} + \alpha \underline{I})^{-1}$$

- Matrix inverse is now better conditioned and unique

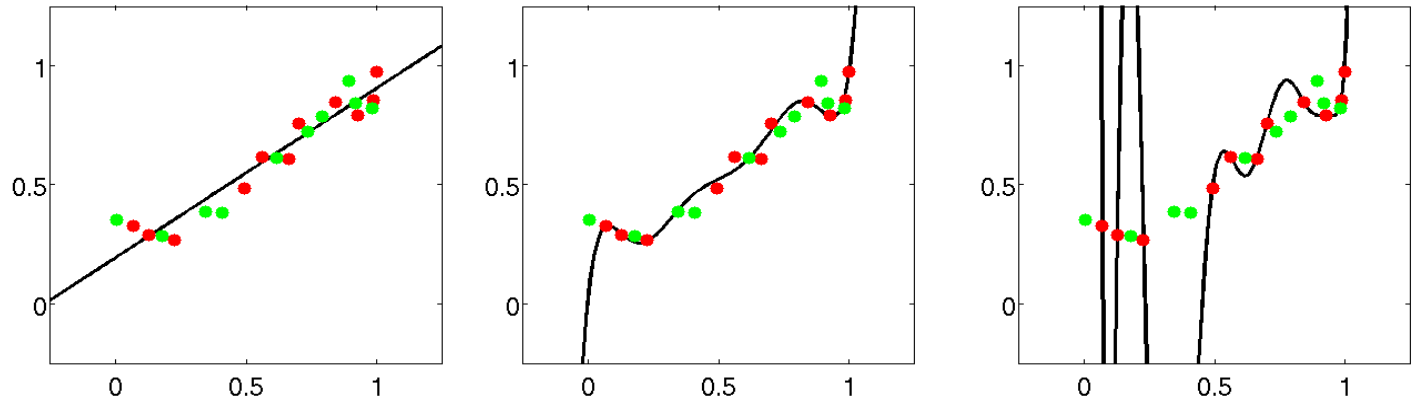
Notes:

- “Shrinks” the parameters θ toward zero
- Large α : we prefer small θ to small MSE
- Regularization term is independent of the data
 - reduces our model variance, can increase bias

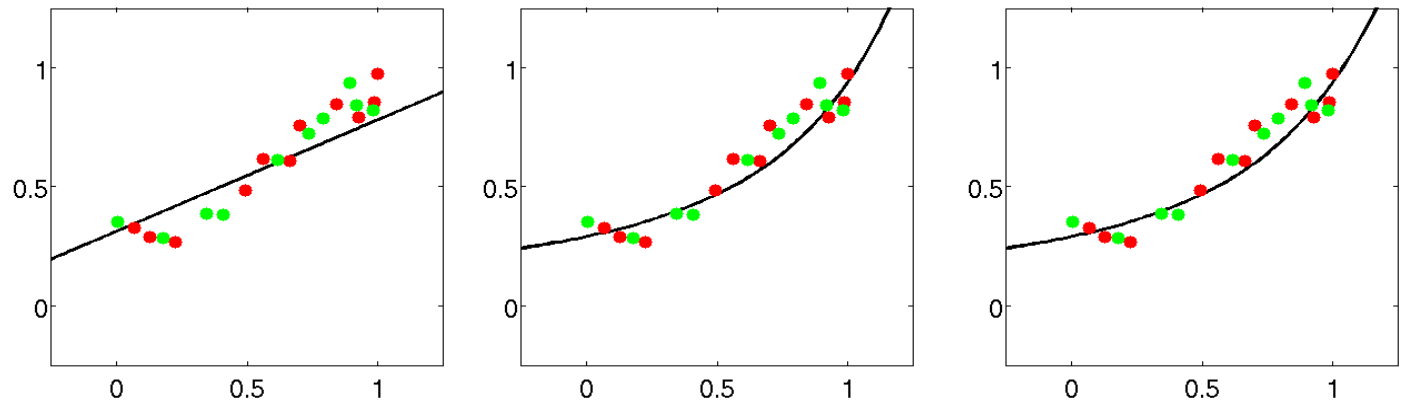
Regularization

Compare between unreg. & reg. results

$\alpha = 0$
(Unregularized)



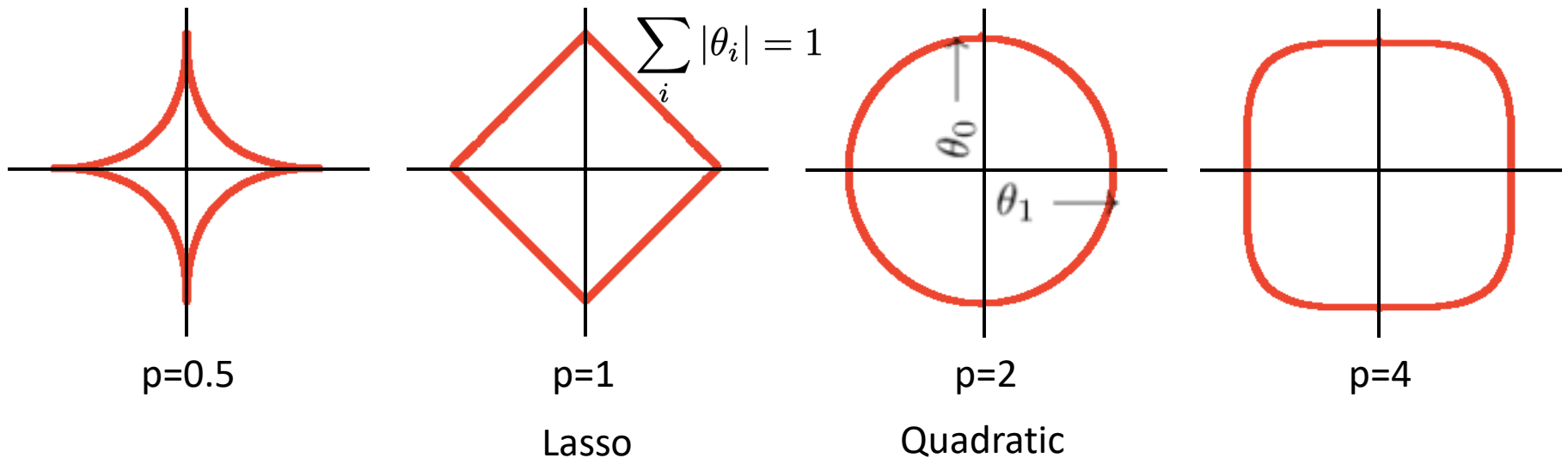
$\alpha = 1$



Different regularization functions

More generally, for the L_p regularizer: $\left(\sum_i |\theta_i|^p \right)^{\frac{1}{p}}$

Isosurfaces: $\|\theta\|_p = \text{constant}$

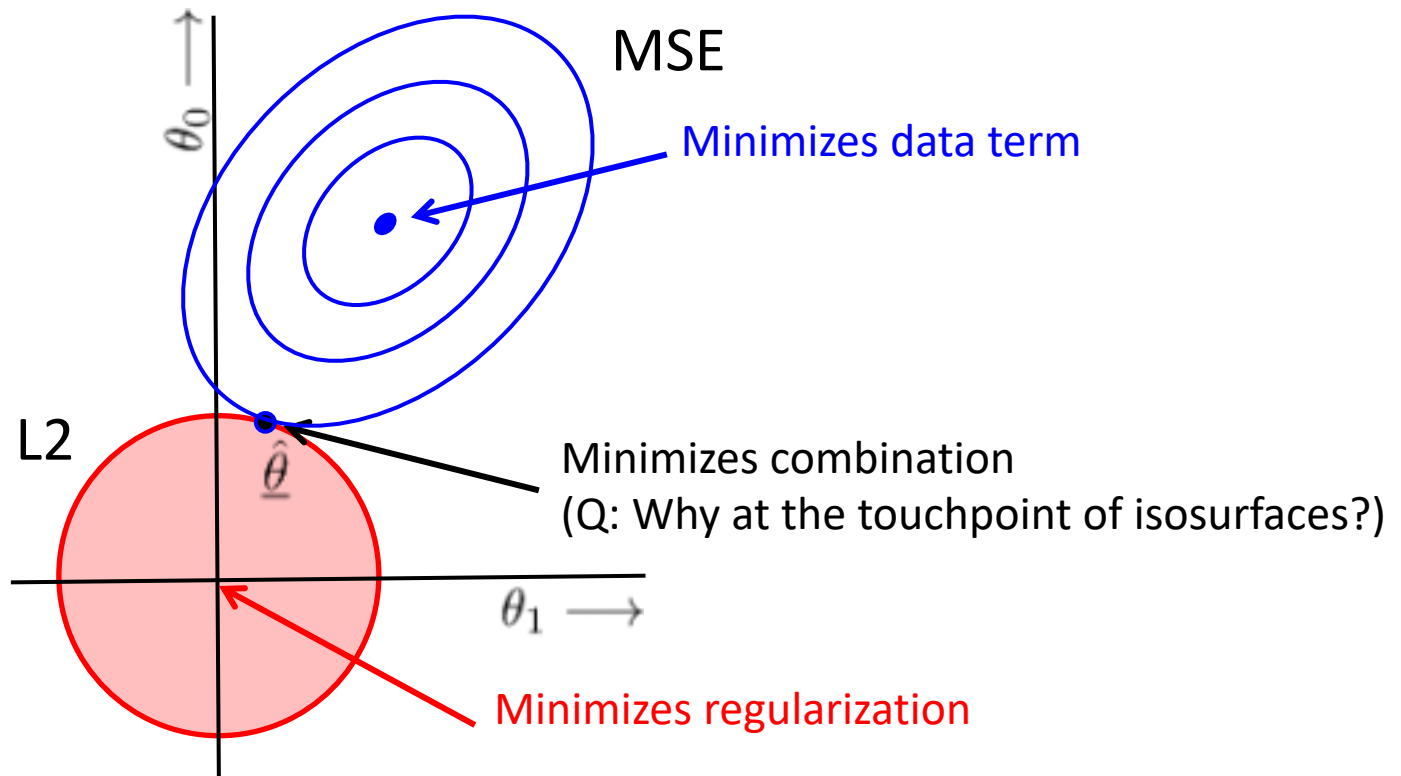


L_0 = limit as $p \rightarrow 0$: “number of nonzero weights”, a natural notion of complexity

L_∞ = limit as $p \rightarrow \infty$: “maximum parameter value”

Regularization: L1 vs L2

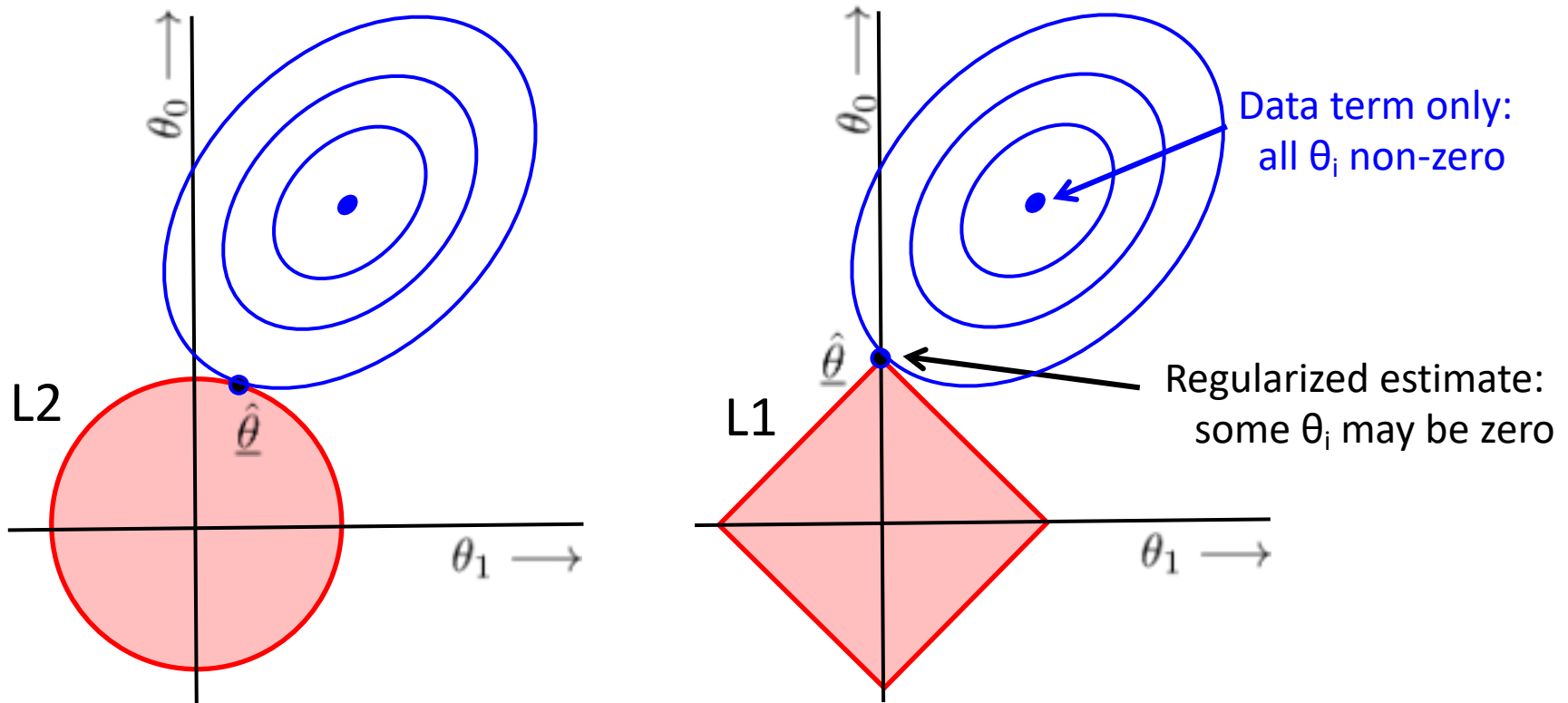
Estimate balances data term & regularization term



Regularization: L1 vs L2

Estimate balances data term & regularization term

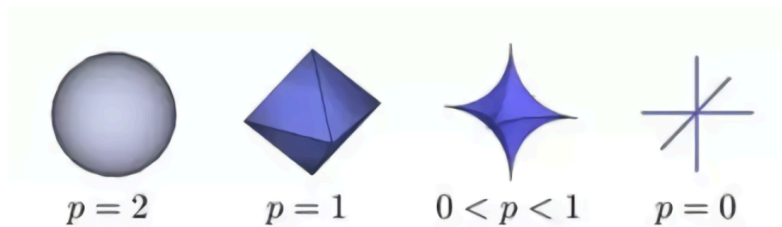
Lasso tends to generate sparser solutions than a quadratic regularizer.



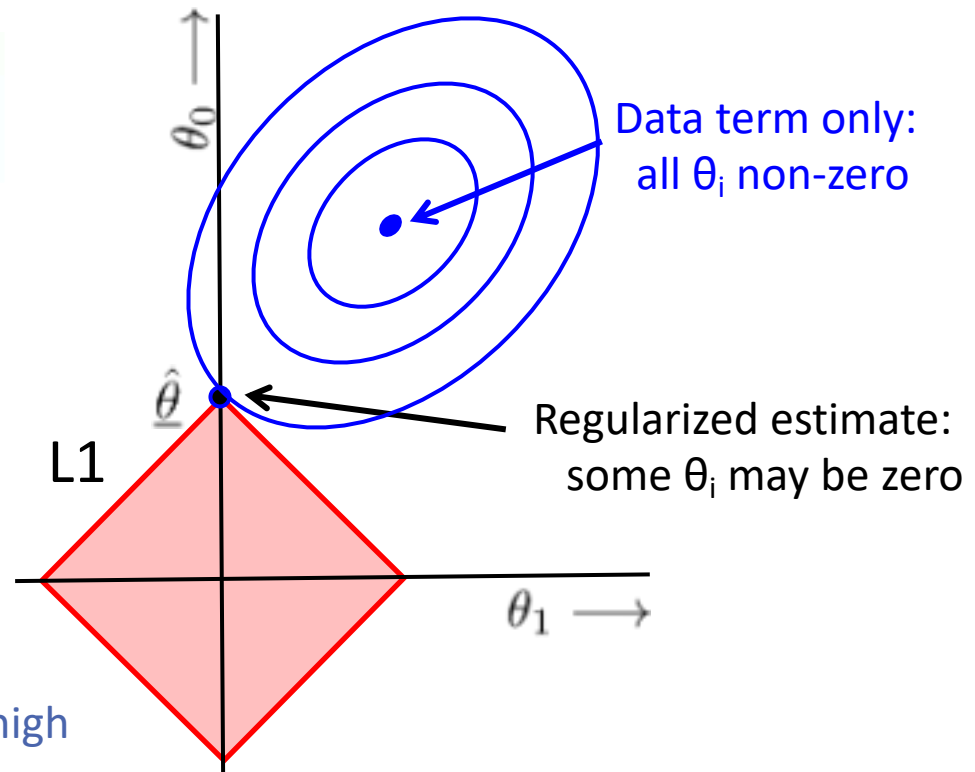
Regularization: L1 vs L2

Estimate balances data term & regularization term

Lasso tends to generate sparser solutions than a quadratic regularizer.



Probability that touch point is an edge is high



Machine Learning

Non-linear Regression

Linear Classification: Perceptron

Cross-Validation

Linear Classification: Perceptron

Model selection

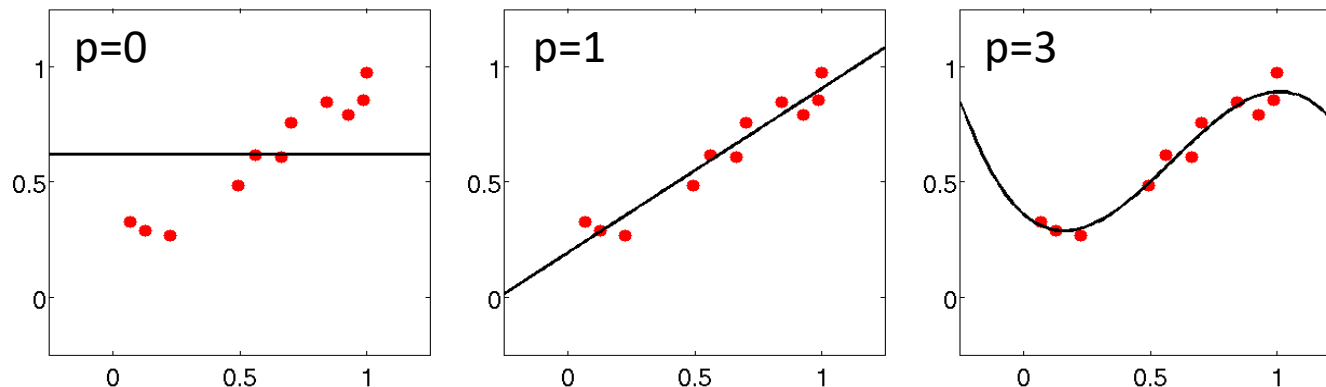
Which of these models fits the data best (e.g., polynomials p)?

- $p=0$ (constant); $p=1$ (linear); $p=3$ (cubic); ...

Or, should we use KNN? Other methods?

Model selection problem

- Can't use training data to decide



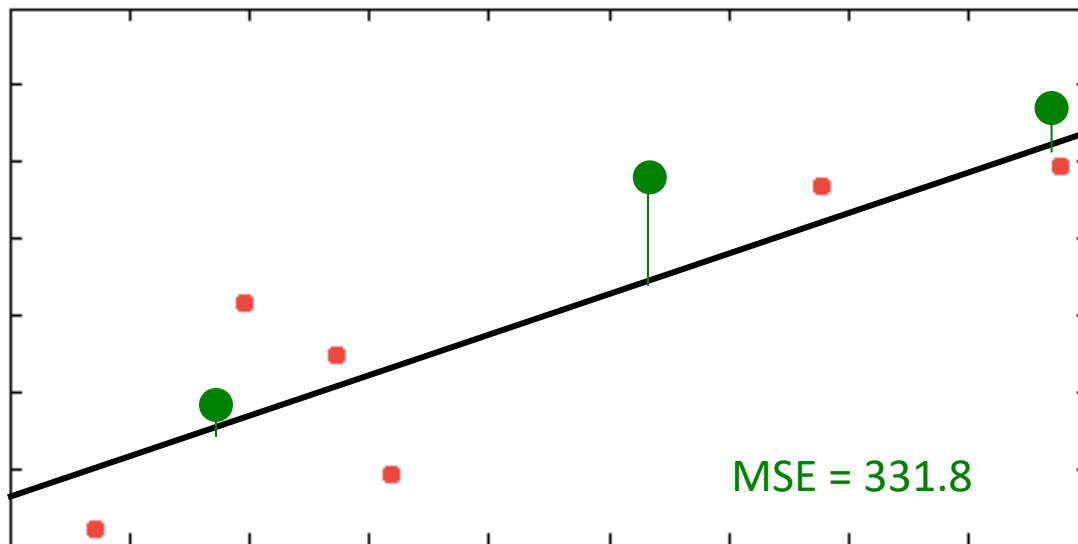
Hold-out method

Validation data

- “Hold out” some data for evaluation (e.g., 70/30 split), train on remainder

Some problems, if we have few data:

- Few data in hold-out: noisy estimate of the error
- More hold-out data leaves less for training!



Training
data

Validation
data

$x^{(i)}$	$y^{(i)}$
88	79
32	-2
27	30
68	73
7	-16
20	43
53	77
17	16
87	94

K-fold cross-validation method

Advantage: makes best use of your data set by splitting your data multiple times

$n = 8$



Test



Train

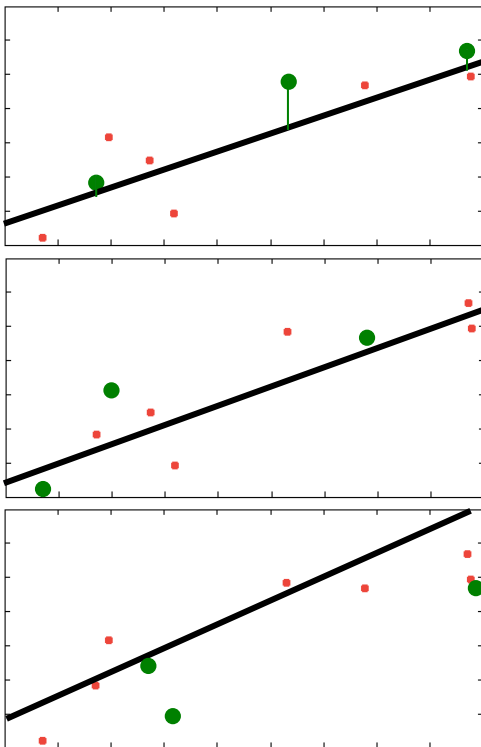
Model 1



K-fold cross-validation method

Advantage: makes best use of your data set by splitting your data multiple times

- Divide data into, e.g., $K=3$ disjoint sets
- Hold out one set for evaluation, Train on other sets. Then average predictions.



Split 1:
MSE = 331.8

Split 2:
MSE = 361.2

Split 3:
MSE = 669.8

➔ 3-Fold X-Val MSE
= 464.1

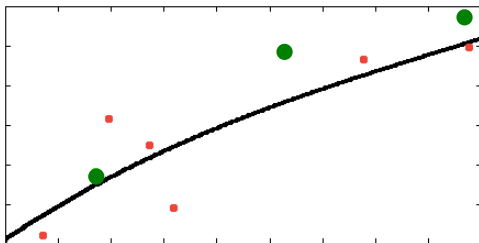
Training
data
Validation
data

$x^{(i)}$	$y^{(i)}$
88	79
32	-2
27	30
68	73
7	-16
20	43
53	77
17	16
87	94

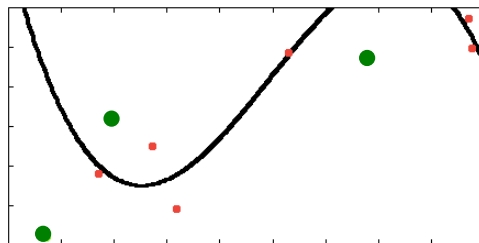
Cross-validation method

K-fold cross-validation

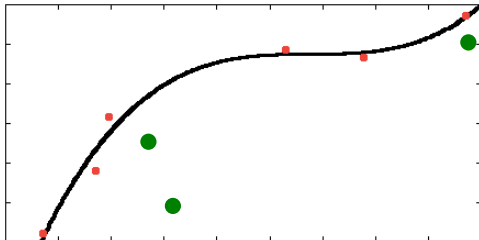
- Divide data into, e.g., $K=3$ disjoint sets
- Hold out one set for evaluation, Train on other sets



Split 1:
MSE = 280.5



Split 2:
MSE = 3081.3



Split 3:
MSE = 1640.1



3-Fold X-Val MSE
= 1667.3

Training
data
Validation
data

$x^{(i)}$	$y^{(i)}$
88	79
32	-2
27	30
68	73
7	-16
20	43
53	77
17	16
87	94

Cross-validation

Advantages

- Lets us use more (m) validation data
- (= less noisy estimate of test performance)

Disadvantages

- More work
 - Trains K models instead of just one
- Doesn't evaluate any particular predictor
 - Evaluates K different models & averages
 - Scores hyperparameters / procedure, not an actual, specific predictor!

Also: still estimating error for $m' < m$ data...

Learning curves

Plot performance as a function of training size

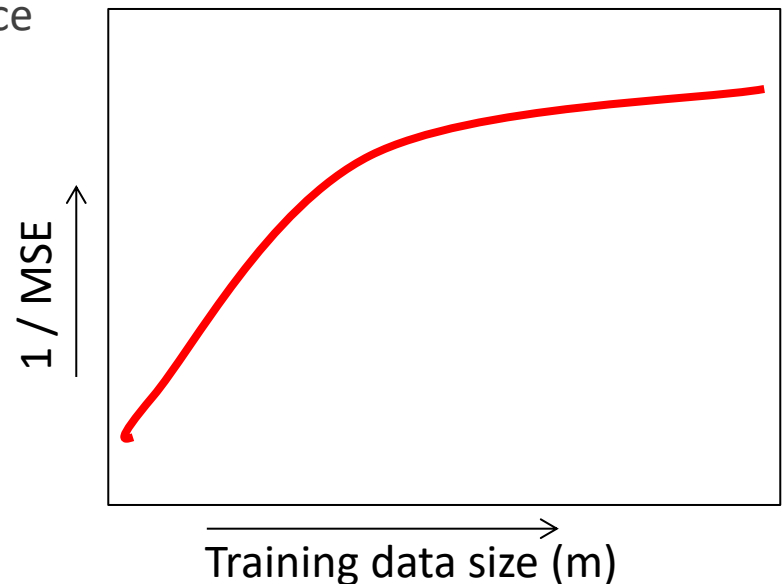
- Assess impact of fewer data on performance
- Ex: MSE0 - MSE (regression)
- or 1-Err (classification)

Few data

- More data significantly improve performance

“Enough” data

- Performance saturates

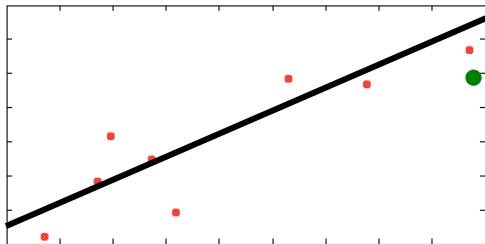


If slope is high, decreasing m (for validation) might have a big impact...

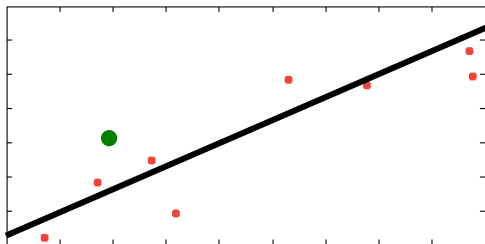
Leave-one-out cross-validation

When $K=m$ (# of data), we get

- Train on all data except one, Evaluate on the left-out data
- Repeat m times (each data point held out once) and average



MSE = ...



MSE = ...

⋮

Training
data
Validation
data

$x^{(i)}$	$y^{(i)}$
88	79
32	-2
27	30
68	73
7	-16
20	43
53	77
17	16
87	94

➡ LOO X-Val MSE
= ...

Cross-validation Issues

Need to balance

- Computational burden (multiple trainings)
- Accuracy of estimated performance / error

Single hold-out set

- Estimates performance with $m' < m$ data
- Need enough data to trust performance estimate
- Estimates performance of a particular, trained learner

K-fold XVal

- K times as much work, computationally
- Better estimates, still of performance with $m' < m$ data

LOO XVal

- m times as much work, computationally
- $m' \approx m$

Machine Learning

Non-linear Regression

Linear Classification: Perceptron

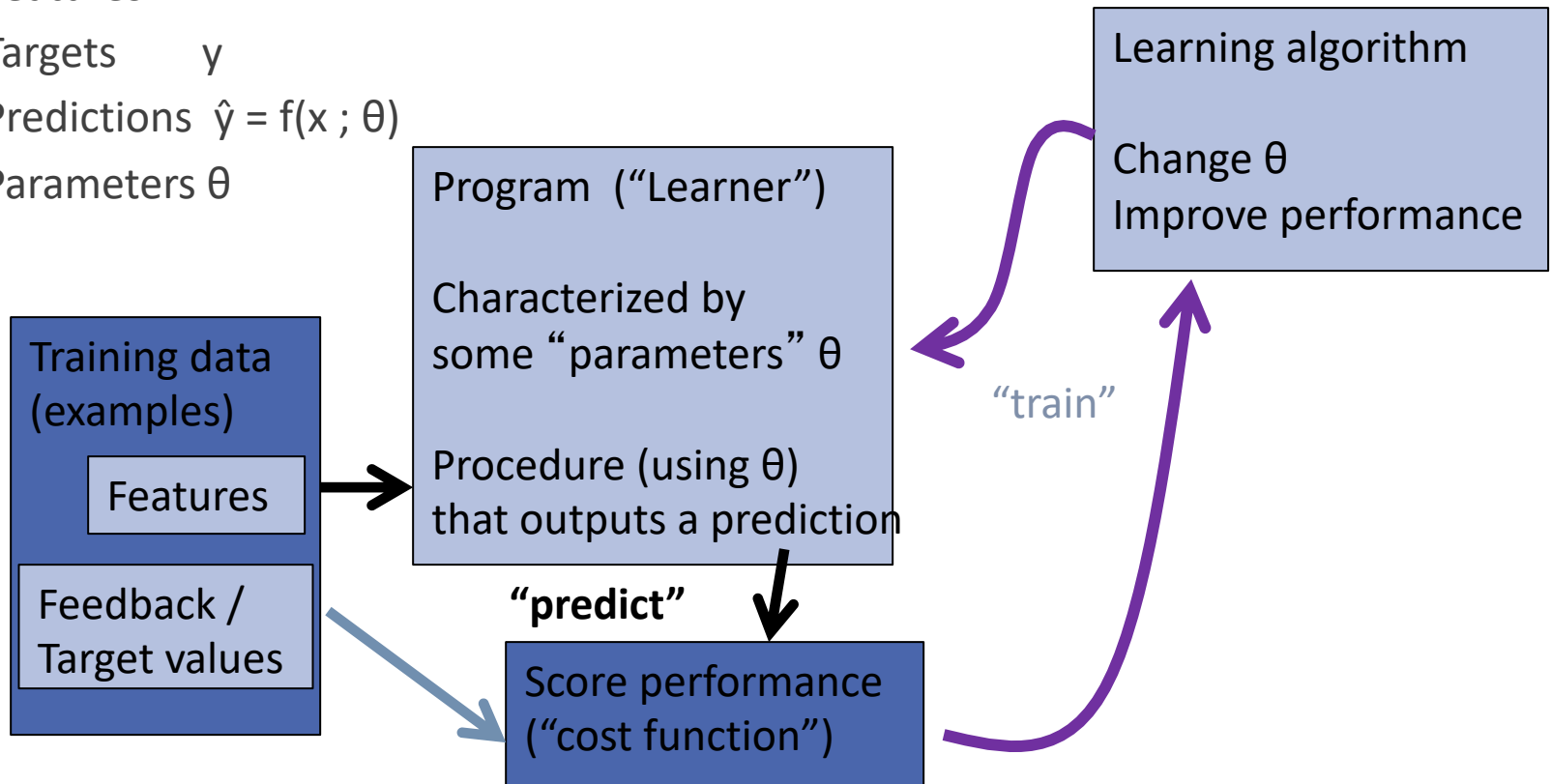
Cross-Validation

Linear Classification: Perceptron

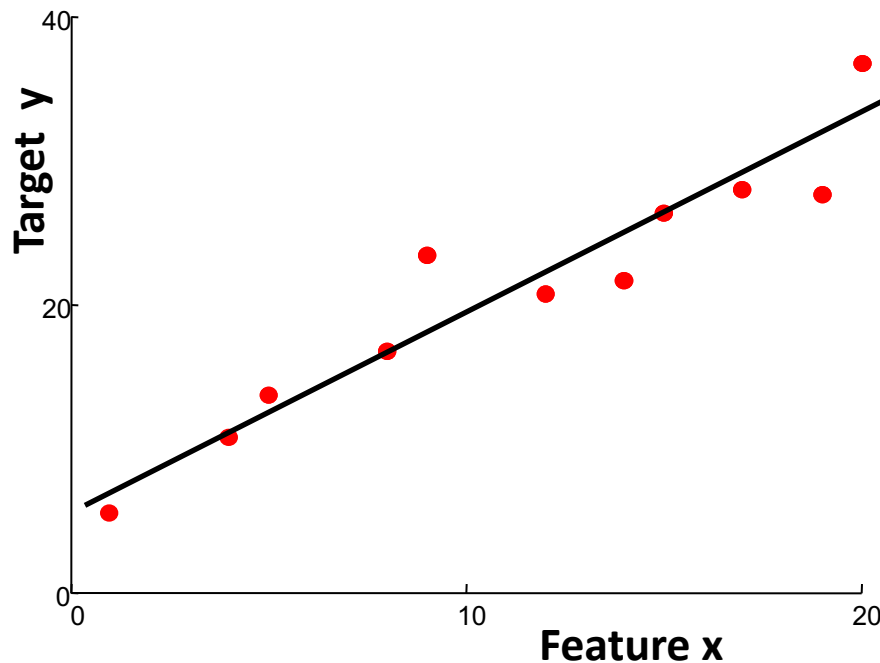
Supervised learning

Notation

- Features x
- Targets y
- Predictions $\hat{y} = f(x; \theta)$
- Parameters θ



Linear regression



“Predictor”:

Evaluate line:

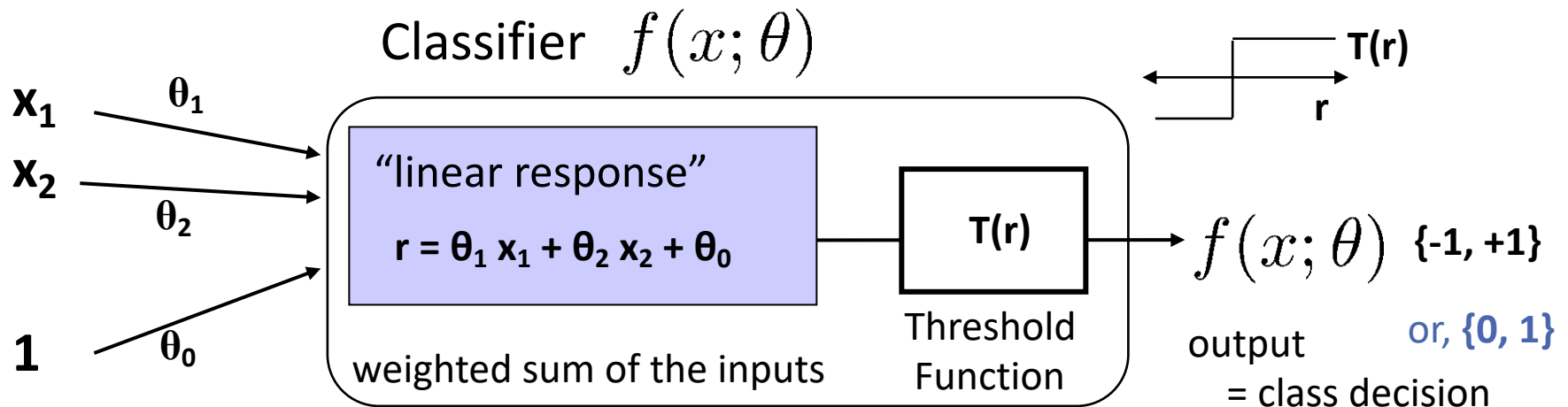
$$r = \theta_0 + \theta_1 x_1$$

return r

Contrast with classification

- Classify: predict discrete-valued target y
- Initially: “classic” binary $\{-1, +1\}$ classes; generalize later

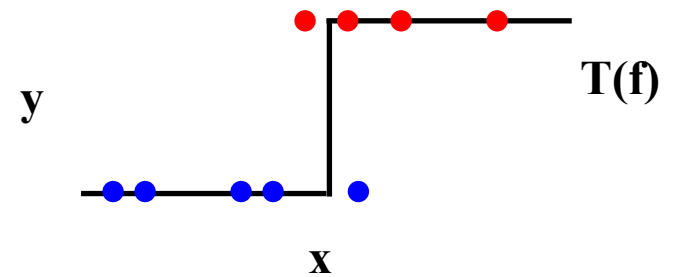
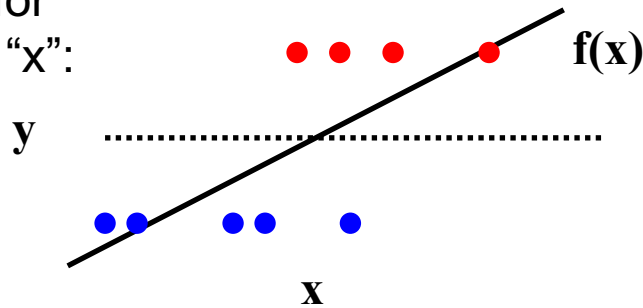
Perceptron Classifier



```
r = X.dot( theta.T );           # compute linear response
Yhat = (r > 0)                   # predict class 1 vs 0
Yhat = 2*(r > 0)-1               # or "sign": predict +1 / -1
# Note: typically convert classes to "canonical" values 0,1,...
# then convert back ("learner.classes[c]") after prediction
```

Perceptron Classifier

Visualizing for
one feature "x":



Perceptrons

Perceptron = a linear classifier

- The parameters θ are sometimes called weights (“w”)
 - real-valued constants (can be positive or negative)
- Input features $x_1 \dots x_n$; define an additional constant input “1”

A perceptron calculates 2 quantities:

1. A weighted sum of the features, **the response**

$$r = \theta \cdot \Phi(x) = \sum_i \theta_i \Phi_i(x)$$

2. This sum is then thresholded by the $T(\cdot)$ function, **the prediction**

$$T(r) = \hat{y}(x)$$

Perceptron: a simple artificial model of human neurons

- weights = “synapses”
- threshold = “neuron firing”

Perceptron Decision Boundary

The perceptron is defined by the decision algorithm:

$$f(x; \theta) = \begin{cases} +1 & \text{if } \theta \cdot x^T > 0 \\ -1 & \text{otherwise} \end{cases}$$

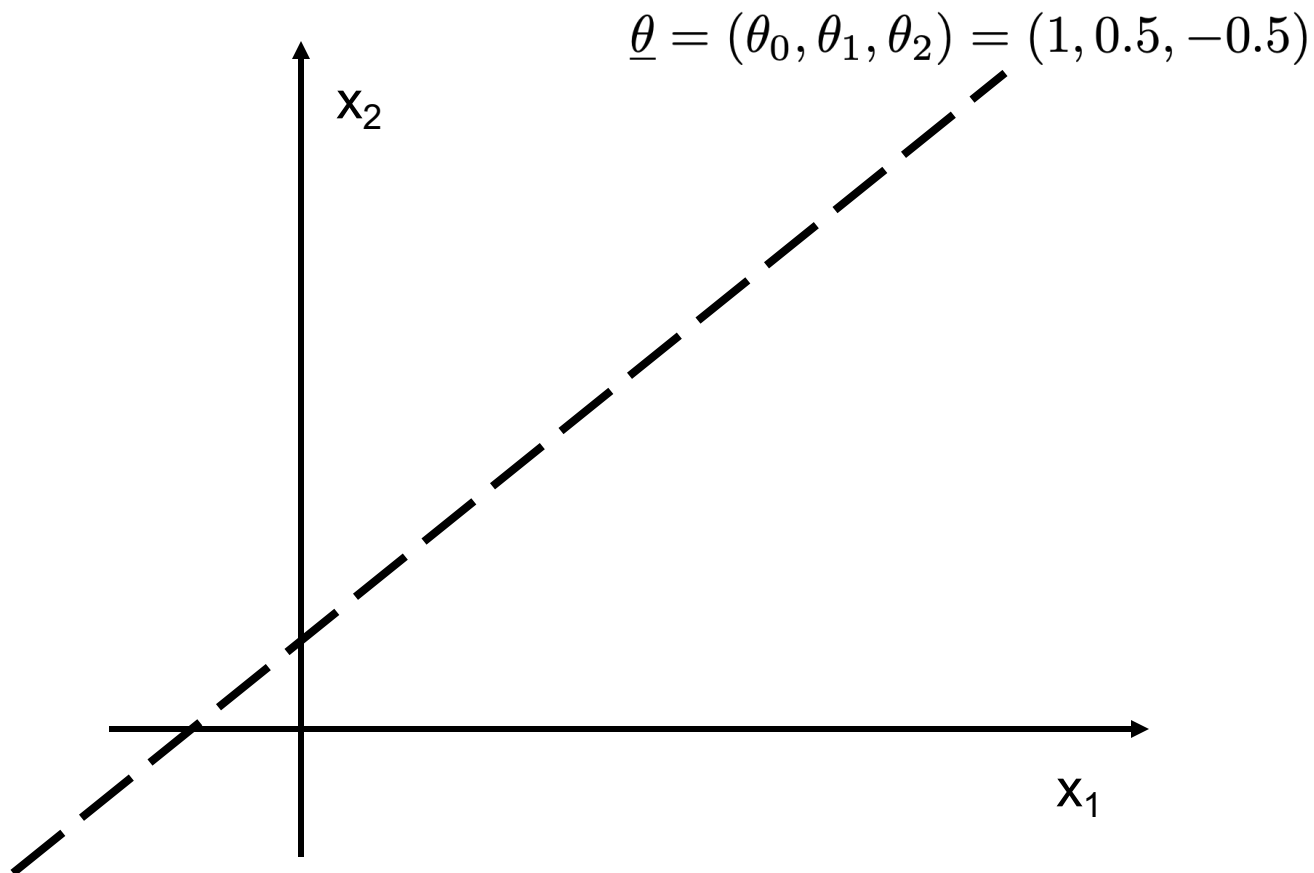
Perceptron represents a hyperplane decision surface in d-dimensional space

- A line in 2D, a plane in 3D, etc.

The equation of the hyperplane is given by $\theta \cdot x^T = 0$

This defines the set of points that are exactly on the boundary.

Example: Linear Decision Boundary



Example: Linear Decision Boundary

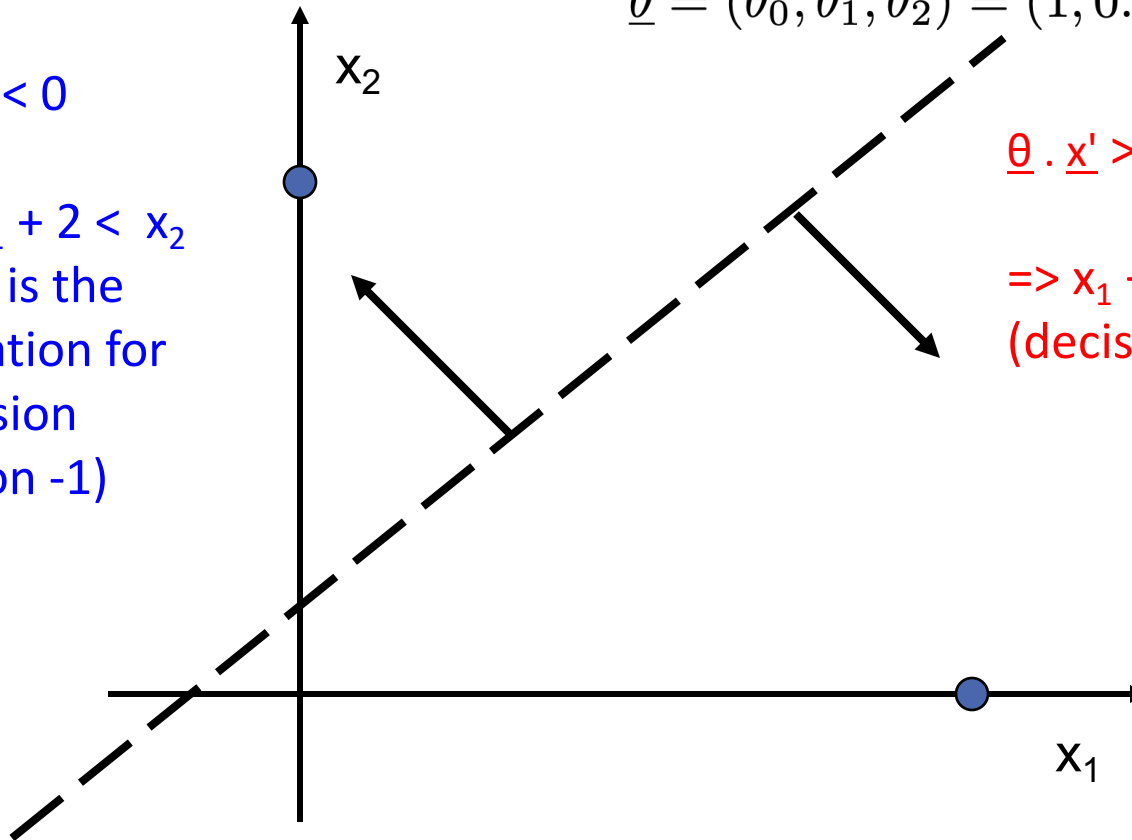
$$\underline{\theta} = (\theta_0, \theta_1, \theta_2) = (1, 0.5, -0.5)$$

$$\underline{\theta} \cdot \underline{x}' < 0$$

$\Rightarrow x_1 + 2 < x_2$
(this is the
equation for
decision
region -1)

$$\underline{\theta} \cdot \underline{x}' > 0$$

$\Rightarrow x_1 + 2 > x_2$
(decision region +1)



Example: Linear Decision Boundary

